Einsum Trees

An Abstraction for Optimizing the Execution of Tensor Expressions

Breuer · Blacher · Engel · Giesen · Heinecke · Klaus · Remke

CS259::Paul Talma
April 20, 2026

Overview

Background and Motivation

Einsum Notation

A compact notation for **pure tensor operations**:

$$\texttt{einsum}(I_1, \ldots, I_n \to O\,;\; T_1, \ldots, T_n)$$

Examples:

Expression	**Operation**	**Name**
`einsum(pq, qr -> pr; A, B)`	$O_{pr} = \sum_q A_{pq} B_{qr}$	Matrix multiply $A \cdot B$
`einsum(pq, pq -> pq; A, B)`	$O_{pq} = A_{pq} B_{pq}$	Hadamard product $A \odot B$
`einsum(pq, qr -> r; A, B)`	$O_r = \sum_{pq} A_{pq} B_{qr}$	$= \sum_q B_{qr} \sum_p A_{pq}$

Rough semantics: indices in inputs but not outputs are contracted (sum-of-products).

Attractions of Einsum

Concise
Literate
Efficient

```python
class ScaledDotProductAttention(nn.Module):
    ''' Scaled Dot-Product Attention '''

    def __init__(self, temperature, attn_dropout=0.1):
        super().__init__()
        self.temperature = temperature
        self.dropout = nn.Dropout(attn_dropout)
        self.softmax = nn.Softmax(dim=2)

    def forward(self, q, k, v, mask=None):

        attn = torch.bmm(q, k.transpose(1, 2))
        attn = attn / self.temperature

        if mask is not None:
            attn = attn.masked_fill(mask, -np.inf)

        attn = self.softmax(attn)
        attn = self.dropout(attn)
        output = torch.bmm(attn, v)

        return output, attn


class MultiHeadAttentionOld(nn.Module):
    ''' Multi-Head Attention module '''

    def __init__(self, n_head, d_model, d_k, d_v, dropout=0.1):
        super().__init__()

        self.n_head = n_head
        self.d_k = d_k
        self.d_v = d_v

        self.w_qs = nn.Linear(d_model, n_head * d_k)
        self.w_ks = nn.Linear(d_model, n_head * d_k)
        self.w_vs = nn.Linear(d_model, n_head * d_v)
        nn.init.normal_(self.w_qs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_ks.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_vs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_v)))

        self.attention = ScaledDotProductAttention(temperature=np.power(d_k, 0.5))
        self.layer_norm = nn.LayerNorm(d_model)

        self.fc = nn.Linear(n_head * d_v, d_model)
        nn.init.xavier_normal_(self.fc.weight)

        self.dropout = nn.Dropout(dropout)


    def forward(self, q, k, v, mask=None):

        d_k, d_v, n_head = self.d_k, self.d_v, self.n_head

        sz_b, len_q, _ = q.size()
        sz_b, len_k, _ = k.size()
        sz_b, len_v, _ = v.size()

        residual = q

        q = self.w_qs(q).view(sz_b, len_q, n_head, d_k)
        k = self.w_ks(k).view(sz_b, len_k, n_head, d_k)
        v = self.w_vs(v).view(sz_b, len_v, n_head, d_v)

        q = q.permute(2, 0, 1, 3).contiguous().view(-1, len_q, d_k) # (n*b) x lq x dk
```

```python
class MultiHeadAttentionNew(nn.Module):
    def __init__(self, n_head, d_model, d_k, d_v, dropout=0.1):
        super().__init__()
        self.n_head = n_head

        self.w_qs = nn.Linear(d_model, n_head * d_k)
        self.w_ks = nn.Linear(d_model, n_head * d_k)
        self.w_vs = nn.Linear(d_model, n_head * d_v)

        nn.init.normal_(self.w_qs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_ks.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_k)))
        nn.init.normal_(self.w_vs.weight, mean=0, std=np.sqrt(2.0 / (d_model + d_v)))

        self.fc = nn.Linear(n_head * d_v, d_model)
        nn.init.xavier_normal_(self.fc.weight)
        self.dropout = nn.Dropout(p=dropout)
        self.layer_norm = nn.LayerNorm(d_model)

    def forward(self, q, k, v, mask=None):
        residual = q
        q = rearrange(self.w_qs(q), 'b l (head k) -> head b l k', head=self.n_head)
        k = rearrange(self.w_ks(k), 'b t (head k) -> head b t k', head=self.n_head)
        v = rearrange(self.w_vs(v), 'b t (head v) -> head b t v', head=self.n_head)
        attn = torch.einsum('hblk,hbtk->hblt', [q, k]) / np.sqrt(q.shape[-1])
        if mask is not None:
            attn = attn.masked_fill(mask[None], -np.inf)
        attn = torch.softmax(attn, dim=3)
        output = torch.einsum('hblt,hbtv->hblv', [attn, v])
        output = rearrange(output, 'head b l v -> b l (head v)')
        output = self.dropout(self.fc(output))
        output = self.layer_norm(output + residual)
        return output, attn
```

The Problem

Einsum notation specifies what to compute, not how.

Different implementations vary widely in their efficiency.

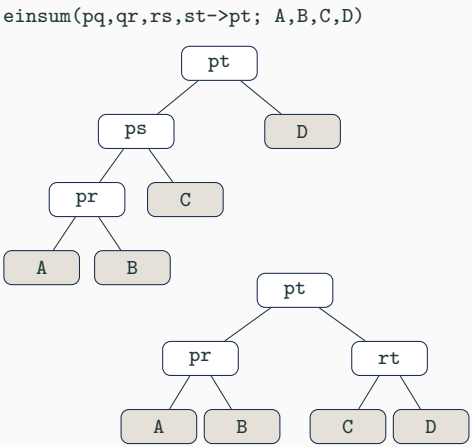
How can we efficiently compile einops into efficient implementations?

Contraction Paths

- An n-ary einsum decomposes into binary contractions
- Different orderings: same result, different flop counts/max intermediate tensor size
- Finding the optimal path is NP-hard. Good heuristics exist: cotengra, opt_einsum
- The chosen schedule is the **contraction path**

Contraction Paths

- An n-ary einsum decomposes into binary contractions
- Different orderings: same result, different flop counts/max intermediate tensor size
- Finding the optimal path is NP-hard. Good heuristics exist: cotengra, opt_einsum
- The chosen schedule is the **contraction path**



Tensor Processing Primitives (TPPs)

A small set of hand-optimized kernels.

The main primitive is GEMM: `C[N][M] += A[K][M] * B[N][K]`. (Also: batched ("packed") GEMM, transpose.)

- Efficient because: cache-blocked tiling keeps the working set in L1/L2, SIMD vectorizes the inner K loop, register reuse amortizes memory latency
- Requires K, M, N to be **rightmost in memory**. Row-major storage: last index in the index string is the unit-stride (fast) dimension
- Loop dimensions outside the GEMM are iterated with OpenMP. Must be large enough to saturate cores, small enough to keep the GEMM tile in cache

Motivation

Two Problems, Solved in Isolation

Contraction path optimization

- Minimizes flops or peak intermediate size
- Assumes fixed row-major layout, ignores data movement

Local binary lowering

- Optimizes layout for one contraction at a time
- Unaware of the layout produced by the preceding step

Consequence

Unnecessary tensor transpositions between contractions.
A full memory read and write per contraction step.

Joint optimization over the full contraction path is the opportunity.

What Goes Wrong Without Joint Optimization

Task: `einsum(pq, qr, rs -> ps)`, path $(A \cdot B) \cdot C$

Local (isolated) approach:

- Step 1: optimize $A \cdot B$ locally. Natural output layout: `pr` ($p{=}M$, $r{=}N$, $q{=}K$).
- Step 2: need `einsum(pr, rs -> ps)`. $K{=}r$ must be rightmost in I_1. Have `pr`. Need `rp`. Transpose required.

Joint approach:

- Step 1 produces layout `rp` instead.
- Step 2: $I_1 =$ `rp`. Condition satisfied. No transpose.

```
Step 2 GEMM condition:

I1 must end with KM. K = r, M = p.

Local result:  I1 = pr <- wrong order Transpose:
pr -> rp <- full memory pass

Joint result:  I1 = rp <- correct order No
transpose needed.  ok

Cost of one transposition:  O(|p| * |r|) memory
reads+writes.  Paid at every contraction step if
layouts are chosen in isolation.
```

The Einsum Trees Solution

Main Contributions

Einsum Tree IR

An intermediate representation encoding both the contraction path and the memory layout of every intermediate tensor, globally.

Optimization algorithm

Three simple tree transformations applied in a single deterministic preorder traversal. No search, no autotuning. Compile time under 4 ms.

`einsum_ir`: open-source C++ implementation

Backend: LIBXSMM + OpenMP. MIT license. Evaluated on Arm (Grace), x86 server (Xeon), x86 desktop (Ryzen).

How the Pieces Fit Together

- The IR gives the optimizer a **global view** of layout decisions across the full contraction path
- Both optimization and lowering are deterministic single-pass algorithms. No tuning loop.
- Dimension taxonomy (R/C/M/N/K) governs the lowering at every node

Dimension Taxonomy

Type	Appears in	Role	Example (einsum(pq,qr→pr))
K	Both inputs, not output	GEMM K (contraction)	q
M	Left input + output	GEMM M	p
N	Right input + output	GEMM N	r
C	Both inputs + output	Packed GEMM batch dim	p in `einsum(pqr,psr->pqs)`
R	One input, not output	Unary reduction	q in `einsum(pq,p->p)`

- Dimensions not mapped to a GEMM type become **loop dimensions**: iterated with OpenMP
- Key constraint: K, M, N, C must occupy the rightmost positions in each index string

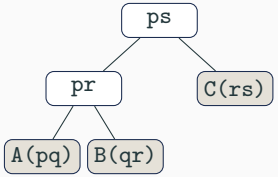
Einsum Tree IR

Contraction tree (baseline):

- Rooted binary tree. Leaves: input tensors. Root: output.
- Encodes contraction order, abstracts irrelevant details

Einsum Tree IR adds:

- **Dimension ordering** at each node. The index string *is* the memory layout declaration.
- Each dimension labeled K/M/N/C/loop
- Unary nodes for **permutations** (transpose primitive)




```
einsum(pq, qr, rs -> ps)
```

GEMM Mapping Condition

For binary node $\texttt{einsum}(I_1,\ I_2 \to O)$ to map to a (packed) GEMM:

$$
\begin{aligned}
I_1 &: \quad \ldots\ g_K\ g_M\ g_C \\
I_2 &: \quad \ldots\ g_N\ g_K\ g_C \\
O &: \quad \ldots\ g_N\ g_M\ g_C
\end{aligned}
$$

g_K, g_M, g_N: contiguous blocks of K/M/N-type dims. g_C: batch dims.

- Remaining dims become **loop dimensions**, parallelized with OpenMP
- When the mapping fails, a **transpose** node is inserted. The optimizer's goal: satisfy this condition at every node without unnecessary transpositions

Optimizing Einsum Trees

Three transformations:

- **Swap.** Exchange left and right children
- **Reorder.** Permute the index string at a node
- **Insert.** Add a unary permutation node (transpose primitive)

Procedure:

- Preorder traversal. At each binary node: classify dims, apply transformations until GEMM mapping satisfied
- Leaf mismatches realized as transpose primitives

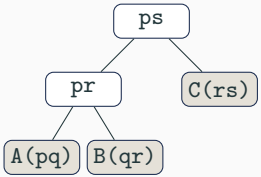
No search required
Transformations applied greedily at each node. Single pass over the tree.

Compile time:

Approach	Time
ET-TPP (this work)	$<$**4 ms**
ET-TBLIS	$<$2 ms
OE-Torch	$<$1 ms
TVM-Ansor	10^3–10^4 s

Toy Example: Initial Tree and Root Analysis

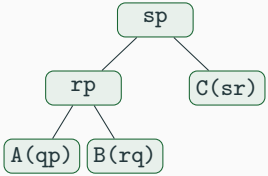
`einsum(pq, qr, rs -> ps)`, path $(A \cdot B) \cdot C$




```
Root node:  einsum(pr, rs -> ps)

Classify dims:  p:  left + output only -> M r:  both
inputs, not output -> K s:  right + output only -> N

GEMM condition needs:  I1 ends with KM = rp (have:
pr) x I2 ends with NK = sr (have:  rs) x O ends with
NM = sp (have:  ps) x

-> Reorder all three index strings.
```

Toy Example: Optimized Tree

After full optimization:




```
Recurse into child:  einsum(qp, rq -> rp)

p:  M, q:  K, r:  N

I1 ends with KM = qp ok I2 ends with NK = rq ok O
ends with NM = rp ok

Child output rp = root's I1 requirement.  No
transpose node needed anywhere.  ok
```

- `rp` satisfies the child's GEMM condition *and* the parent's I_1 requirement simultaneously
- Local approach: produces `pr`, forces a transpose at the next step

Evaluation

Results: Single Binary Contractions (TCCG)

[Insert Fig. 3 from paper]
Grace testbed: GFLOPS vs. TCCG setting ID
ET-TPP · ET-TBLIS · TVM-Ansor · ATen · OE-Torch

- 24 benchmark settings. ET-TPP competitive across all three testbeds.
- TVM-Ansor slightly ahead on a subset, but requires hours of autotuning vs. milliseconds for ET-TPP.
- Settings 17–18: small loop dimensions limit OpenMP parallelism. A known weakness of the greedy heuristic.
- ATen and OE-Torch (TTGT-style local lowering) are consistently slower. The gap quantifies avoided transposition overhead.

Results: Complex Contraction Trees

Grace testbed, FP32 GFLOPS

Workload	ET-TPP	Best other	Speedup
SYN	4838	3172 (Ansor)	**1.5**×
TT	3315	2455 (TBLIS)	**1.4**×
FCTN	3092	2578 (Ansor)	**1.2**×
TRN	1963	933 (TBLIS)	**2.1**×
MERA	4025	1958 (TBLIS)	**2.1**×
TNLM	4378	3532 (Torch)	**1.4**×

[Insert Figs. 3–5 for full per-testbed plots]

- ET-TPP outperforms all four baselines on most workloads.
- TVM-Ansor: N/A for MERA, TNLM. OOM or timeout after 12 hours.
- TW on SPR: ET-TBLIS wins. Small h-dimension limits vectorization.

Discussion

Critical Evaluation

Comparison to TTGT

- TTGT (used by ATen, partially by OE-Torch): permute inputs to GEMM layout, call GEMM, permute output back. Always pays 2–3 transpositions per contraction.
- ET-TPP's advantage is exactly the elimination of these transpositions. The speedup over ATen/OE-Torch directly quantifies this benefit.

TVM-Ansor

- Ansor searches a larger space. ET-TPP beats it anyway on most settings.
- Ansor is unusable for MERA and TNLM (OOM/timeout). ET-TPP compiles both in

FP32 only

- Modern ML runs BF16/FP16/INT8. Generalizability asserted in §8, not demonstrated.
- Heuristic dimension-size bounds were tuned for FP32. Need re-derivation for lower precision.

Transformer attention is absent

- Attention: $\mathrm{softmax}(QK^\top/\sqrt{d}) \cdot V$. The softmax breaks the contraction chain. Cannot be expressed as einsum.
- ET-TPP can optimize $QK^\top$ and $\cdot V$ separately but not fused. Flash Attention, which fuses both to avoid materializing the $O(n^2)$ attention matrix, is out of scope.

Extensions and Open Questions

MLIR integration
A future Einsum Tree MLIR dialect lowering to a TPP dialect would make this approach composable with existing compiler pipelines. The authors flag this as the natural next step.

GPU / Triton backend
The IR is backend-agnostic. A Triton lowering would extend the approach to GPUs, but requires a new cost model (warp occupancy, shared memory tiling) and faces strong baselines in cuTENSOR.

Learned cost model
The heuristic dimension-size bounds were hand-tuned by benchmarking. A learned model could generalize across hardware without manual re-tuning. The IR's explicit layout representation makes feature extraction straightforward.